Experiment No.09

Aim:Building a Basic K-Means Clustering Model to Detect Unusual Network Traffic Patterns patterns using an open-access dataset

In [11]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

In [12]:
```
train = pd.read_csv("UNSW_NB15_training-set.csv")
test = pd.read_csv("UNSW_NB15_testing-set.csv")
```

In [13]:
```
df = pd.concat([train, test], ignore_index=True)
```

In [5]:
```
print("Training dataset shape:", train_df.shape)
print("Testing dataset shape:", test_df.shape)
```

```
Training dataset shape: (82332, 45)
Testing dataset shape: (175341, 45)
```

In [14]:
```
features = [
    'sbytes', 'dbytes', 'dur',
    'spkts', 'dpkts',
    'sload', 'dload'
]
```

In [16]:
```
features = [col for col in features if col in df.columns]
```

In [18]:
```
X = df[features].apply(pd.to_numeric, errors='coerce')
```

In [19]:
```
X = X.replace([np.inf, -np.inf], np.nan)
```

In [20]:
```
X = X.fillna(X.median())
```

In [21]:
```
X = X.dropna()
```

In [22]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [23]:
```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
```

In [24]:
```
distances = np.linalg.norm(
    X_scaled - kmeans.cluster_centers_[clusters],
    axis=1
)
```

In [25]:
```
threshold = np.percentile(distances, 95)
anomalies = distances > threshold
```

In [26]:
```
results = X.copy()
results['cluster'] = clusters
results['distance'] = distances
results['anomaly'] = anomalies
```

In [27]:
```
print("Cluster counts:")
print(results['cluster'].value_counts().sort_index())

print("\nAnomaly counts:")
print(results['anomaly'].value_counts())
```

```
Cluster counts:
cluster
0    245022
1     12536
2       115
Name: count, dtype: int64

Anomaly counts:
anomaly
False    244789
True      12884
Name: count, dtype: int64
```

In [28]:
```
print("\nPotential unusual network traffic:")
print(
    results[results['anomaly']]
    .sort_values('distance', ascending=False)
    .head(10)
)
```

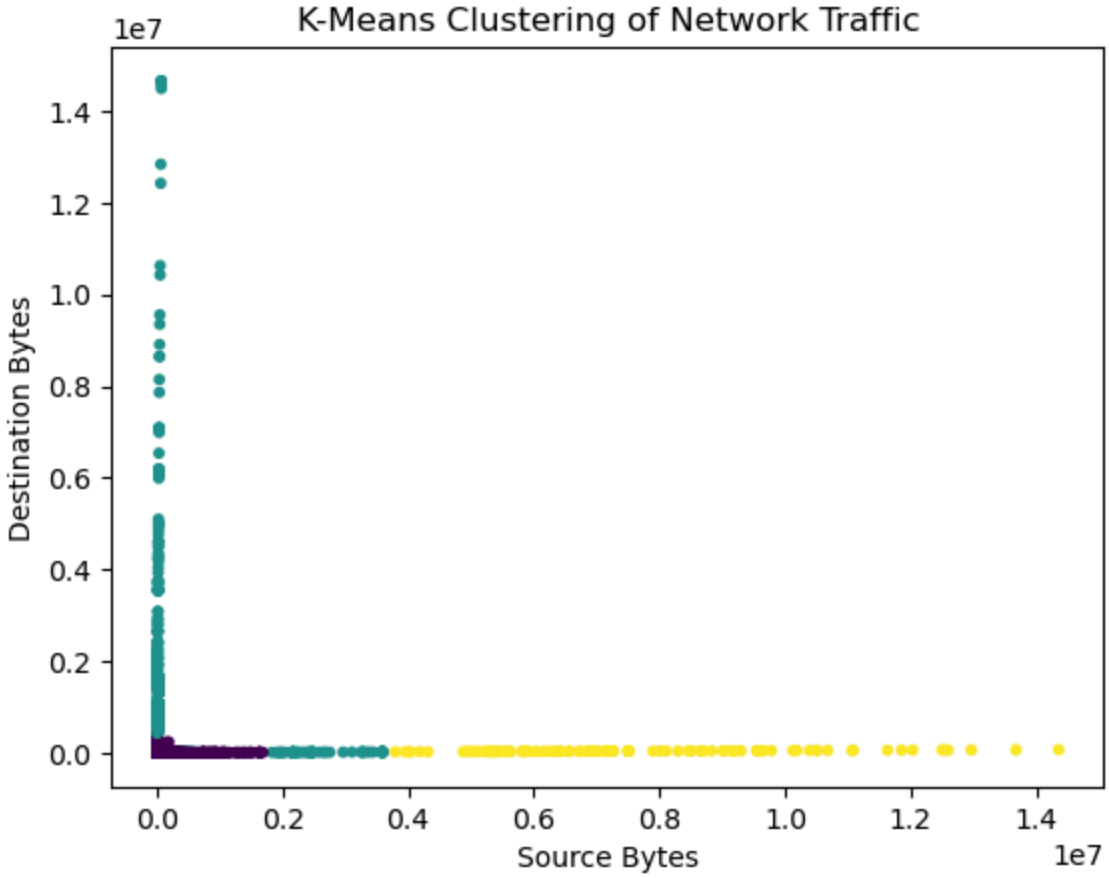
```
Potential unusual network traffic:
        sbytes    dbytes        dur  spkts  dpkts         sload        dload
\
515      70700  14657531  57.237827   1642  11018   9875.567383  2048463.625
154820   71234  14655550  57.953392   1654  10974   9827.345703  2022896.625
44143    70256  14655417  54.265602   1632  10970  10351.013670  2160349.250
10554    70688  14590859  59.983105   1642  10872   9421.986328  1945816.875
165270   70506  14491793  59.962872   1638  10850   9400.883789  1933257.375
203432   62790  12838547  59.844826   1458   9660   8387.959961  1716067.250
182178   61248  12422665  59.972408   1422   9346   8164.421387  1656940.125
10154    51734  10625785  59.966820   1192   7958   6895.946777  1417377.125
11401    50882  10422875  59.946285   1178   7728   6784.607422  1390782.000
49377    46794   9554162  59.971622   1082   7090   6236.416016  1274311.375

        cluster    distance  anomaly
515           1  138.893423     True
154820        1  138.626294     True
44143         1  138.545790     True
10554         1  137.696843     True
165270        1  137.075486     True
203432        1  121.554879     True
182178        1  117.569711     True
10154         1  100.119695     True
11401         1   97.706454     True
49377         1   89.497519     True
```

In [29]:
```
plt.scatter(
    results['sbytes'],
```

```
    results['dbytes'],
    c=results['cluster'],
    cmap='viridis',
    s=10
)

plt.xlabel("Source Bytes")
plt.ylabel("Destination Bytes")
plt.title("K-Means Clustering of Network Traffic")
plt.show()
```




In [ ]: